1.1. Menger's Theorem

We have the following nice results:

Theorem 1. Menger's theorem (vertex version). Let $G = (V, E)$ be an graph (directed or undirected) and let $s, t \in V$ be two distinct *nonadjacent* vertices. Then the size of the minimum s $-t$ vertex cut is equal to the maximum number of pairwise vertex-disjoint paths from s to t.

Here, "vertex-disjoint" means they don't share any vertices other than s or t. Note that "nonadjacent" is important, otherwise it will be impossible to cut s from t by removing vertices.

Theorem 2. Menger's theorem (edge version). Let $G = (V, E)$ be a graph (directed or undirected) and let $s, t \in V$ be two distinct vertices. Then the size of the minimum s–t edge cut is equal to the maximum number of pairwise edge-disjoint paths from s to t.

These theorems were proved by Karl Menger, an Austrian–American mathematician, in 1927.

Exercise 3. Prove that the undirected version of Theorem 1 or Theorem 2 follows from the directed version. (Hint: Convert each undirected edge to two directed edges)

We show by construction that the undirected version of Menger's theorem, both vertex and edge, follows from the directed version.

Given an undirected graph G, replace each undirected edge (u,v) with two directed edges: u->v and v->u. Call this directed graph G'.

Any vertex-disjoint s-t paths in G correspond directly to vertex-disjoint s-t paths in G', just by following each edge in its direction. Conversely, any vertex-disjoint directed s-t paths in G' correspond to vertex-disjoint undirected s-t paths in G, by ignoring directions.

A set of vertices whose removal disconnects s and t in G also disconnects them in G' as removing vertices remove all incident edges in both directions. So the converse also holds: removing vertices in G' affects G identically. In G', the size of the minimum directed s-t vertex cut equals the maximum number of vertex-disjoint directed s-t paths. By the correspondence above, this equality translates back to G.

For the edge version, the same transformation works, edge-disjoint undiected paths correspond to edge-disjoint directed paths, using the two directed edges per undirected edge, and vice versa by ignoring the direction of the edges. So, edge cuts correspond similarly.

Thus, the undirected version follows directly from the directed version.

Exercise 4. Prove that the directed version of Theorem 1 follows from the directed version of Theorem 2.

Let G be a directed graph with nonadjacent s, t.

By directed edge Menger, the minimum s-t edge cut equals the maximum number of edge-disjoint s-t paths.

I claim that "any set of vertex-disjoint paths is also edge-disjoint". To prove this, we prove the contrapositive statement: "If two paths are not edge-disjoint, then they are not vertex-disjoint." That is, there are two paths who share an edge, that must also share vertices. Suppose two paths, S_1 and S_2, share an edge. Then, they must also share the endpoints of said edge, u and v. Then, these two paths, in fact, are also not vertex-disjoint.

Therefore, any set of vertex-disjoint paths are also edge-disjoint. Therefore, max vertex-disjoint paths <= max edge-disjoint paths = min edge cut.

But any s-t vertex cut is also an edge cut, since removing vertices removes all incident edges. So the min edge cut <= min vertex cut.

This also implies that max vertex-disjoint paths <= min vertex cut. The reverse (max vertex-disjoint paths >= min vertex cut) also works, since the cut must hit each path. Therefore, the equality holds: Theorem 1 follows from the directed version of Theorem 2.

Exercise 7. Prove that two nodes x and y are 2-edge-connected if there is a path from x to y that does not pass through a bridge.

[=>] If x and y are 2-edge-connected, then they remain connected after the removal of any single edge. In particular, remove any bridge b from G. Since x and y remain connected, there is an x-y path avoiding b. Doing this for every bridge yields a path that avoids all bridges.

[<=] Suppose there exists an x-y path P with no bridges. Take an arbitrary edge e and consider the following cases:

[Case 1: e is in path P] Since e is not a bridge, removing e leaves some alternative x-y path.

[Case 2: e is not in path P] then P still connects x and y after removing e.

Thus, x and y remain connected after any single edge removal, so they are 2-edge-connected.

Exercise 8. Prove that G' is 2-edge-connected. G' is made from a directed graph that is strongly connected to an undirected graph where we ignore the directions. (Hint: Use Exercise 7 and show that there is no bridge.)

Assume for contradiction, that G' has a bridge e = {u,v}. Removing e disconnects G' into two components.

In the original directed graph G, all edges are oriented. Since G is strongly connected, there is a directed path from u to v and v to u. These paths must use edges other than e, since e can only be used in one direction in each path. Thus, in G' there exist two edge-disjoint u-v paths, one in each direction's underlying undirected edges, contradicting that e is a bridge.

Therefore, G' has no bridges. By Exercise 7, a connected graph with no bridges is 2-edge-connected.

Exercise 10: Find an O(n+e)-time algorithm that computes a strongly-connected orientation of a connected undirected graph, or determines if there is none, using a single DFS traversal.

First, we check 2-edge-connectivity by running DFS to compute bridges in G via disc and low. If any bridge is found, we output None. This is correct because by Robbins' theorem, 2-edge-connectivity is necessary for this to work.

Second, we orient edges during another DFS. We perform DFS from an arbitrary root r. We orient each tree edge downward, from parent to child. Then, orient each back edge upward, from descendant to ancestor. We output the resulting directed graph. The orientation produced is exactly the one proven to be strongly connected in the lecture. Every vertex can reach r via back edges since low[v] <= disc[parent] due to no bridges, and r can reach every vertex via tree edges.

Running DFS two times still results in 2 * O(n + e) = O(n + e) time complexity.

Exercise 11. Let G be a connected graph with n >= 3 nodes. Prove that G is biconnected if and only if it is 2-connected.

- A graph is **biconnected** if it is connected and has no articulation points.
- A graph is **2-connected (2-vertex-connected)** if it has more than 2 vertices and remains connected after the removal of any single vertex.

[=>] Suppose G is biconnected with n >= 3. Take any vertex v and remove it, giving us G - v. If G - v were disconnected, then v would have been an articulation point, contradicting the biconnectivity of the graph. Thus, G - v is connected for all v, so G is 2-connected.

[<=] Suppose G is 2-connected with n >= 3. By definition, removing any single vertex leaves the graph connected. Thus, no vertex is an articulation point, so G is biconnected.

Therefore, the two properties are equivalent for n >= 3.

Exercise 12. Modify the code of Hopcroft-Tarjan BCC to account for self-loops. (Hint: Get all the self-loops first).

Before running DFS, we identify the self-loops (v,v) in the graph. Push each self-loop onto the edge stack immediately when encountered, treating it as its own BCC. During DFS, when exploring neighbors, we skip self-loops, since they don't affect lowlink computation and would cause issues with parent-child relationships. After DFS completes, for each self-loop (v,v), output it as a separate BCC, a single edge component.

This works because self-loops are biconnected components of size 1, an edge whose removal disconnects nothing, but they don't affect vertex connectivity. The standard algorithm assumes distinct endpoints, handling them separately avoids incorrect lowlink computations.

```
// Before DFS
for each edge (u,v) in graph:
        if u == v:
                push (u,v) onto stack
                pop and output as BCC

// During DFS
if v == u: continue
```

```
def compute_BCCs(undirected graph G = (V, E))
    for each i in V
        disc[i] := −1
    mark every edge as not visited
    time := 0
    edge_stack := (an empty stack)
    bccs := (an empty list of lists)

    def get_bcc(edge e)   // pop until e
        bcc := (an empty list)
        while e is not the last element of bcc
            bcc.append(edge_stack.pop())
        bccs.append(bcc)
    def dfs(node i, edge e)
        low[i] := disc[i] := time
        time := time + 1
        for each edge e′ ∈ adj[i]
            Let j be the other endpoint of e′
            if e′ is not yet visited
                mark e′ as visited
                edge_stack.push(e′)
            if disc[j] = −1
                dfs(j, e′)
                low[i] := min(low[i], low[j])
                if low[j] ≥ disc[i]
                    get_bcc(e′)
            else if e ≠ e′   // don't use the same tree edge
                low[i] = min(low[i], disc[j])

    for each v in V
        if disc[s] = −1
            dfs(s, null)

    return bccs
```

Exercise 13. Prove that the block-cut tree T (articulation points and BCCs as nodes, edges for containment) is acyclic.

Assume, for contradiction, that T contains a cycle C.

In C, articulation points and BCC nodes alternate, since edges only connect an articulation point to a BCC. We follow the cycle in G b starting at some articulation point a_1 in BCC B_1, move to B_1's other articulation point a_2 via a path inside B_1, from a_2 go to BCC B_2, then to a_3, etc.

This constructs a cycle in the original graph G that passes through multiple BCCs. However, BCCs are maximal biconnected subgraphs connected only through articulation points, a cycle passing through multiple BCCs would create additional connections between them, contradicting maximality. In other words, C would have formed a whole BCC in the first place.

Therefore, no cycle exists in T, so T is acyclic. Since G is connected, T is connected, and T is also a tree.

Exercise 14. Prove that the leaves in the modified block-cut tree represents exactly the following:

- **A node of G that is not an articulation point**
- **A BCC that is a self-loop**

Let T' be the modified block-cut tree. We consider the following cases:

[Case 1: leaf = non-articulation point]
A vertex v that is not an articulation point belongs to exactly one BCC. In T', v connects only to that BCC-node, thus degree 1. In other words, a leaf.

[Case 2: leaf = self-loop] By Exercise 12, we can make self-loops into their own BCCs. Since it is a self-loop, it contains exactly one vertex v, the endpoint, and no articulation points. In T', this BCC-node connects only to v. Thus, degree 1, thus a leaf.
[Case 3: There are no other leaves]
We show that there are no other leaves in T'.

- If an articulation point belongs to >= 2 BCCs, then it has a degree >= 2 in T'.
- A non-self-loop BCC with >= 2 vertices contains at least two vertices or one vertex with multiple edges. Since BCC is maximal, it connects to at least one articulation point, or multiple vertices, thus degree >= 2.
- A BCC that is a single edge between two distinct vertices connects to both endpoints, thus, it has degree 2.

Therefore, leaves are exactly non-articulation points, and self-loop BCCs.

Theorem 15. A graph is 2-edge-connected if and only if it has an ear decomposition.

[=>] Let G be 2-edge-connected. We start with a single vertex. Since G has no bridges, we repeatedly find an ear, a maximal path whose internal vertices have degree 2 in the current subgraph, and add it. We continue until all edges are included. The process terminates because each step adds at least one new edge.

[<=] Suppose G has an ear decomposition. We start from the initial cycle, which is 2-edge-connected. Adding an ear preserves that 2-edge-connectivity as any edge in the new ear lies on a cycle. That being, the ear plus a path between its endpoints in the existing graph, so no bridge is introduced. Therefore, the final graph is 2-edge-connected.

Theorem 16: (Whitney, 1932) A graph with at least two edges is 2-vertex-connected if and only if it has an open ear decomposition (every ear is open, except the first one, which is a cycle).

[=>] Let G be 2-connected. We start with any cycle, which exists since the minimum degree of all vertices is 2. We repeatedly find an open ear, since removing any vertex leaves G connected, there exists a path between two distinct vertices of the current subgraph whose internal vertices are new and have degree 2 when added. We add it, and continue until all edges are continued.

[<=] Suppose G has an open ear decomposition. The base cycle is 2-connected. Adding an open ear preserves that 2-connectivity. Endpoints are distinct and already in the graph, so no new articulation point is created, internal vertices have degree 2 and are not cut vertices. Thus, the final graph is 2-connected.